**Workshop 1**

*Maria Lucía Castillo García*

# Índice de entregables

**Evidencia** ..... 2

**Parte 1: Construcción de la API** ..... 2

**Parte 2: Dockerfile** ..... 2

**Parte 3: Docker Compose con tres instancias** ..... 4

**Parte 4: Configuración de Nginx** ..... 7

**Parte 5: Evidenciar el balanceamiento** ..... 10

**Preguntas de análisis - Parte 6** ..... 12

**Reto** ..... 15

# Evidencia

```
casti@MLC MINGW64 ~/workshop1-bash
$ cd ~

casti@MLC MINGW64 ~
$ mkdir -p ~/workshop1-docker/api ~/workshop1-docker/nginx

casti@MLC MINGW64 ~
$ cd workshop1-docker/

casti@MLC MINGW64 ~/workshop1-docker
$ ls
api/  nginx/
```

## Parte 1: Construcción de la API

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat api/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

@app.get("/")
def home():
    return jsonify({
        "message": "Hola desde Flask",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

casti@MLC MINGW64 ~/workshop1-docker
$ cat api/requirements.txt
Flask
```

## Parte 2: Dockerfile

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat > api/Dockerfile << 'EOF'
> FROM python:3.12-slim
>
> WORKDIR /app
> COPY requirements.txt .
> RUN pip install --no-cache-dir -r requirements.txt
>
> COPY app.py .
>
> EXPOSE 5000
>
> CMD ["python", "app.py"]
> EOF
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker build -t taller1-api ./api
[+] Building 19.5s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 204B
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
 => => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
 => => sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6c27e03d6b43e7b4eac5f 249B / 249B
 => => sha256:1560cfed01dc4c72c07f789d860078d74466a1f9a26b33a2e35d887b5f90dd3f 12.12MB / 12.12MB
 => => sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075890620e3db64b47bb662433a 4.27MB / 4.27MB
 => => sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 29.79MB / 29.79MB
 => => extracting sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be
 => => extracting sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075890620e3db64b47bb662433a
 => => extracting sha256:1560cfed01dc4c72c07f789d860078d74466a1f9a26b33a2e35d887b5f90dd3f
 => => extracting sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6c27e03d6b43e7b4eac5f
 => [internal] load build context
 => => transferring context: 388B
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:f9ce46efcc01f782eb46e312bb2ef7a54fab0244247f3e916ada10389e58233c
 => => exporting config sha256:383809f531afd0eb3fbbef3f05ee277f002380fad1927a5d49f1304ad3071cbb
 => => exporting attestation manifest sha256:33856951c2214f277e4112a0b46338c5178a7a6c313f62c1ca17b4a4e8894a0a
 => => exporting manifest list sha256:1c28ed5b85a1de950c3190131e2f0023f075e7f2f680eab33f754734926f193a
 => => naming to docker.io/library/taller1-api:latest
 => => unpacking to docker.io/library/taller1-api:latest
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker images

IMAGE                                      ID             DISK USAGE   CONTENT SIZE   EXTRA
design_1-backend:latest                    b7ce9b819836       2GB            506MB
design_1-frontend:latest                   51cade814978      97.7MB         27.1MB
design_2-backend:latest                    e768fc67baf0       2GB            506MB
design_2-frontend:latest                   4ceff697b5c7      97.7MB         27.1MB
design_3-backend:latest                    b84f4580c5ab       2GB            506MB
design_3-frontend:latest                   35d4d8535d45      97.7MB         27.1MB
design_4-backend:latest                    8395a2c6352c       2GB            506MB
design_4-frontend:latest                   cd3241e2c8b5      97.7MB         27.1MB
docker/welcome-to-docker:latest            c4d56c24da4f      22.2MB         6.03MB    U
ejemplo-cmd:latest                         a30a7160aef4       157MB         41.6MB
ejemplo-combinado:latest                   7f54bcab2e11       157MB         41.6MB
ejemplo-entrypoint:latest                  603694ffee4c       157MB         41.6MB
japeto/pujgcc:latest                       9b3d7bbf4103      2.18GB          448MB    U
japeto/so-tools:v64                        7e990f123c8d       2GB            506MB    U
jupyter/scipy-notebook:latest              fca4bcc9cbd4      5.76GB         1.32GB    U
multithreaded-enemy-cars-backend:latest    c5e180b20b31       2GB            506MB
multithreaded-enemy-cars-frontend:latest   10ccec8ae54d      97.7MB         27.1MB
nginx:alpine                               db35bfc6b295      94.2MB         27.2MB
reto-monitor:latest                        10a354bd55d5       157MB         41.6MB
reto-web:latest                            939fed0e6700      93.4MB         26.3MB
taller1-api:latest                         1c28ed5b85a1       208MB         51.2MB
ubuntu-bash-lab:latest                     c4ff932c0a29       162MB           43MB    U
ubuntu:latest                              2260313b31c8       160MB         45.3MB
workshop3-docker-compose-web:latest        747e028671d5      93.4MB         26.3MB
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker run -d --name prueba -p 5000:5000 -e INSTANCE=api-prueba taller1-api
d86be01442876b5338662edf607bb6d1877f99fbcf7f4365f3cfae92650c10ac

casti@MLC MINGW64 ~/workshop1-docker
$ docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
d86be0144287   taller1-api    "python app.py"   4 seconds ago   Up 4 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   prueba

casti@MLC MINGW64 ~/workshop1-docker
$ curl http://localhost:5000/
{"instance":"api-prueba","message":"Hola desde Flask"}
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker rm -f prueba
prueba

casti@MLC MINGW64 ~/workshop1-docker
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

## Parte 3: Docker Compose con tres instancias

```
casti@MLC MINGW64 ~/workshop1-docker
$ touch docker-compose.yml

casti@MLC MINGW64 ~/workshop1-docker
$ notepad docker-compose.yml
```

```yaml
services:
  api1:
    build: ./api
    environment:
      INSTANCE: api-1

  api2:
    build: ./api
    environment:
      INSTANCE: api-2

  api3:
    build: ./api
    environment:
      INSTANCE: api-3

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat docker-compose.yml
services:
  api1:
    build: ./api
    environment:
      INSTANCE: api-1

  api2:
    build: ./api
    environment:
      INSTANCE: api-2

  api3:
    build: ./api
    environment:
      INSTANCE: api-3

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
casti@MLC MINGW64 ~/workshop1-docker
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.45kB 0.0s done
#1 DONE 0.0s

#2 [api3 internal] load build definition from Dockerfile
#2 transferring dockerfile: 204B done
#2 DONE 0.0s

#3 [api3 internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.2s

#4 [api1 internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [api1 internal] load build context
#5 transferring context: 63B done
#5 DONE 0.0s

#6 [api1 1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
#6 resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea 0.0s done
#6 DONE 0.0s

#7 [api2 2/5] WORKDIR /app
#7 CACHED

#8 [api2 3/5] COPY requirements.txt .
#8 CACHED

#9 [api2 4/5] RUN pip install --no-cache-dir -r requirements.txt
#9 CACHED

#10 [api2 5/5] COPY app.py .
#10 CACHED

#11 [api1] exporting to image
#11 exporting layers done
#11 exporting manifest sha256:1cd9e063590c20b6d65fb087854e112b92ba3ed1f1089301e349870938618dca 0.0s done
#11 exporting config sha256:325e47d6f7f212c79ef47ae782d9a3e22b946269f70406875d27a5b67cb07207
#11 exporting config sha256:325e47d6f7f212c79ef47ae782d9a3e22b946269f70406875d27a5b67cb07207 0.0s done
#11 exporting attestation manifest sha256:46ea7d2c6f55a40680e8ae0db8b6f00ac5e9f28b9a52d07a530253a5333ec895
#11 ...

#12 [api3] exporting to image
#12 exporting layers done
#12 exporting manifest sha256:a178f1fcf73bec51e3b57031bdb0c330241345c735125ad0c0621a94dec96b84 0.0s done
#12 exporting config sha256:ef5526c54d82db019d313dcd65461e23009f90af4c93aca4f6d9575721ce26b4 0.0s done
#12 exporting attestation manifest sha256:7b06f8d1a0940f23aec0c0060eeab57ba964174e4b52635d612989ad6bb42aef 0.1s done
#12 exporting manifest list sha256:dafc517ed025ca814c4daea1085d0deac255521d26c5d0672572c814cc4b299a 0.0s done
#12 naming to docker.io/library/workshop1-docker-api3:latest 0.0s done
#12 unpacking to docker.io/library/workshop1-docker-api3:latest 0.0s done
#12 DONE 0.3s

#11 [api1] exporting to image
#11 exporting attestation manifest sha256:46ea7d2c6f55a40680e8ae0db8b6f00ac5e9f28b9a52d07a530253a5333ec895 0.1s done
```

```
#11 [api1] exporting to image
#11 exporting layers done
#11 exporting manifest sha256:1cd9e063590c20b6d65fb087854e112b92ba3ed1f1089301e349870938618dca 0.0s done
#11 exporting config sha256:325e47d6f7f212c79ef47ae782d9a3e22b946269f70406875d27a5b67cb07207
#11 exporting config sha256:325e47d6f7f212c79ef47ae782d9a3e22b946269f70406875d27a5b67cb07207 0.0s done
#11 exporting attestation manifest sha256:46ea7d2c6f55a40680e8ae0db8b6f00ac5e9f28b9a52d07a530253a5333ec895
#11 ...

#12 [api3] exporting to image
#12 exporting layers done
#12 exporting manifest sha256:a178f1fcf73bec51e3b57031bdb0c330241345c735125ad0c0621a94dec96b84 0.0s done
#12 exporting config sha256:ef5526c54d82db019d313dcd65461e23009f90af4c93aca4f6d9575721ce26b4 0.0s done
#12 exporting attestation manifest sha256:7b06f8d1a0940f23aec0c0060eeab57ba964174e4b52635d612989ad6bb42aef 0.1s done
#12 exporting manifest list sha256:dafc517ed025ca814c4daea1085d0deac255521d26c5d0672572c814cc4b299a 0.0s done
#12 naming to docker.io/library/workshop1-docker-api3:latest 0.0s done
#12 unpacking to docker.io/library/workshop1-docker-api3:latest 0.0s done
#12 DONE 0.3s

#11 [api1] exporting to image
#11 exporting attestation manifest sha256:46ea7d2c6f55a40680e8ae0db8b6f00ac5e9f28b9a52d07a530253a5333ec895 0.1s done
#11 exporting manifest list sha256:639794a1cdde9914d4ea40b841c507e90b686f6c727af9faf271980e72139c4c 0.0s done
#11 naming to docker.io/library/workshop1-docker-api1:latest done
#11 unpacking to docker.io/library/workshop1-docker-api1:latest 0.0s done
#11 DONE 0.3s

#13 [api2] exporting to image
#13 exporting layers done
#13 exporting manifest sha256:c73712e81f04f88cd8eb58d0de88cecbce3422b70f0eeadb5d3e401a0fc69884 0.0s done
#13 exporting config sha256:889ecc6f61c2f654d17b18e112810988bc84f45402e8e8740f49ca34a2a7df3e 0.0s done
#13 exporting attestation manifest sha256:467e5a5952549aacec7ac486c6a30b2dc83425997977b5bc18670be256ee1688 0.1s done
#13 exporting manifest list sha256:41cdd1c892eeb39ca07ed93693cb7ed81495fffaebc99ed755fe5acb158f23f8 0.0s done
#13 naming to docker.io/library/workshop1-docker-api2:latest 0.0s done
#13 unpacking to docker.io/library/workshop1-docker-api2:latest 0.0s done
#13 DONE 0.3s

#14 [api3] resolving provenance for metadata file
#14 DONE 0.1s

#15 [api2] resolving provenance for metadata file
#15 DONE 0.1s

#16 [api1] resolving provenance for metadata file
#16 DONE 0.0s
[+] up 8/8
 ✔ Image workshop1-docker-api1         Built
 ✔ Image workshop1-docker-api2         Built
 ✔ Image workshop1-docker-api3         Built
 ✔ Network workshop1-docker_default    Created
 ✔ Container workshop1-docker-nginx-1  Started
 ✔ Container workshop1-docker-api2-1   Started
 ✔ Container workshop1-docker-api1-1   Started
 ✔ Container workshop1-docker-api3-1   Started
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose ps
NAME                    IMAGE                   COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop1-docker-api1-1    workshop1-docker-api1   "python app.py"          api1      47 seconds ago   Up 47 seconds   5000/tcp
workshop1-docker-api2-1    workshop1-docker-api2   "python app.py"          api2      47 seconds ago   Up 47 seconds   5000/tcp
workshop1-docker-api3-1    workshop1-docker-api3   "python app.py"          api3      47 seconds ago   Up 47 seconds   5000/tcp
workshop1-docker-nginx-1   nginx:alpine            "/docker-entrypoint.…"   nginx     47 seconds ago   Up 47 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ curl http://localhost:8080/
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

## Parte 4: Configuración de Nginx

```
casti@MLC MINGW64 ~/workshop1-docker
$ ls
api/  docker-compose.yml  nginx/

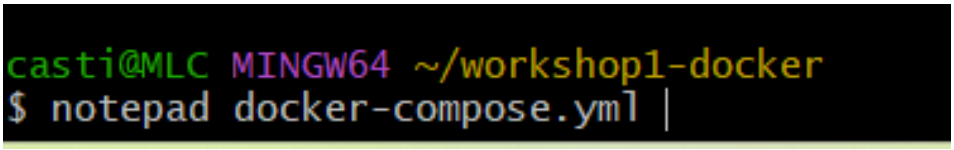
casti@MLC MINGW64 ~/workshop1-docker
$ touch nginx/nginx.conf

casti@MLC MINGW64 ~/workshop1-docker
$ notepad nginx/nginx.conf
```

```
upstream flask_backend {
    server api1:5000;
    server api2:5000;
    server api3:5000;
}

server {
    listen 80;

    location / {
        proxy_pass http://flask_backend;
    }
}
```




```
services:
  api1:
    build: ./api
    environment:
      INSTANCE: api-1

  api2:
    build: ./api
    environment:
      INSTANCE: api-2

  api3:
    build: ./api
    environment:
      INSTANCE: api-3

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api1
      - api2
      - api3
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat nginx/nginx.conf
upstream flask_backend {
    server api1:5000;
    server api2:5000;
    server api3:5000;
}

server {
    listen 80;

    location / {
        proxy_pass http://flask_backend;
    }
}
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose config
name: workshop1-docker
services:
  api1:
    build:
      context: C:\Users\casti\workshop1-docker\api
      dockerfile: Dockerfile
    environment:
      INSTANCE: api-1
    networks:
      default: null
  api2:
    build:
      context: C:\Users\casti\workshop1-docker\api
      dockerfile: Dockerfile
    environment:
      INSTANCE: api-2
    networks:
      default: null
  api3:
    build:
      context: C:\Users\casti\workshop1-docker\api
      dockerfile: Dockerfile
    environment:
      INSTANCE: api-3
    networks:
      default: null
  nginx:
    depends_on:
      api1:
        condition: service_started
        required: true
      api2:
        condition: service_started
        required: true
      api3:
        condition: service_started
        required: true
    image: nginx:alpine
    networks:
      default: null
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose up -d
[+] up 4/4
 ✓ Container workshop1-docker-api3-1  Running
 ✓ Container workshop1-docker-api1-1  Running
 ✓ Container workshop1-docker-api2-1  Running
 ✓ Container workshop1-docker-nginx-1 Started

casti@MLC MINGW64 ~/workshop1-docker
$ docker compose ps
NAME                      IMAGE                  COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop1-docker-api1-1   workshop1-docker-api1  "python app.py"          api1      8 minutes ago    Up 8 minutes    5000/tcp
workshop1-docker-api2-1   workshop1-docker-api2  "python app.py"          api2      8 minutes ago    Up 8 minutes    5000/tcp
workshop1-docker-api3-1   workshop1-docker-api3  "python app.py"          api3      8 minutes ago    Up 8 minutes    5000/tcp
workshop1-docker-nginx-1  nginx:alpine           "/docker-entrypoint.…"   nginx     10 seconds ago   Up 9 seconds    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ curl http://localhost:8080/
{"instance":"api-1","message":"Hola desde Flask"}
```

## Parte 5: Evidenciar el balanceamiento

```
casti@MLC MINGW64 ~/workshop1-docker
$ for i in {1..12}; do curl -s http://localhost:8080/; echo; done
{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}
```

**ACLARACIÓN:**

Se utilizó worker_processes 1 para que fuera más fácil observar el balanceo de carga. Cuando Nginx ejecutaba varios procesos worker, la mayoría de las peticiones terminaban llegando a api-1, lo que ocultaba la rotación esperada. Con un solo worker, las solicitudes se distribuyeron de manera uniforme entre api1, api2 y api3.

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat nginx/nginx.conf
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream flask_backend {
        server api1:5000;
        server api2:5000;
        server api3:5000;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://flask_backend;
        }
    }
}
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat docker-compose.yml
services:
  api1:
    build: ./api
    environment:
      INSTANCE: api-1

  api2:
    build: ./api
    environment:
      INSTANCE: api-2

  api3:
    build: ./api
    environment:
      INSTANCE: api-3

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - api1
      - api2
      - api3
```

## Detener una instancia

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose stop api2
[+] stop 1/1
 ✔ Container workshop1-docker-api2-1 Stopped

casti@MLC MINGW64 ~/workshop1-docker
$ docker compose ps
NAME                       IMAGE                      COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop1-docker-api1-1    workshop1-docker-api1      "python app.py"          api1      34 minutes ago   Up 34 minutes   5000/tcp
workshop1-docker-api3-1    workshop1-docker-api3      "python app.py"          api3      34 minutes ago   Up 34 minutes   5000/tcp
workshop1-docker-nginx-1   nginx:alpine               "/docker-entrypoint.…"   nginx     5 minutes ago    Up 5 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ for i in {1..12}; do curl -s http://localhost:8080/; echo; done
{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}
```

**Recuperación**

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose start api2
[+] start 1/1
 ✔ Container workshop1-docker-api2-1 Started

casti@MLC MINGW64 ~/workshop1-docker
$ docker compose ps
NAME                          IMAGE                     COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop1-docker-api1-1       workshop1-docker-api1     "python app.py"          api1      36 minutes ago   Up 36 minutes   5000/tcp
workshop1-docker-api2-1       workshop1-docker-api2     "python app.py"          api2      36 minutes ago   Up 6 seconds    5000/tcp
workshop1-docker-api3-1       workshop1-docker-api3     "python app.py"          api3      36 minutes ago   Up 36 minutes   5000/tcp
workshop1-docker-nginx-1      nginx:alpine              "/docker-entrypoint.…"   nginx     7 minutes ago    Up 7 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

casti@MLC MINGW64 ~/workshop1-docker
$ for i in {1..12}; do curl -s http://localhost:8080/; echo; done
{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}

{"instance":"api-2","message":"Hola desde Flask"}

{"instance":"api-3","message":"Hola desde Flask"}

{"instance":"api-1","message":"Hola desde Flask"}
```

# Preguntas de análisis - Parte 6

1. **¿Por qué el cliente solamente conoce `localhost:8080`?**

**-** Porque ese es el puerto que Nginx expone hacia afuera. Desde el punto de vista del usuario, todo el sistema parece una sola aplicación ubicada en `localhost:8080`.

Las APIs (`api1`, `api2` y `api3`) también tienen puertos, pero esos puertos solo existen dentro de la red interna creada por Docker Compose. Por eso el cliente nunca se comunica directamente con ellas.

Así que básicamente, Nginx actúa como la recepción de un edificio en donde análogamente todas las personas entran por la misma puerta y él se encarga de enviarlas a la oficina correcta. Así, el cliente no necesita saber cuántas APIs existen ni cuál responderá la petición.

2. **¿Qué función cumple upstream?**

- El bloque upstream es la lista de servidores a los que Nginx puede enviar las peticiones. En lugar de decirle a Nginx "envía todo a api1", se le dice "envía las peticiones al grupo flask_backend". Dentro de ese grupo están api1, api2 y api3.

Gracias a esto, si se agregan o eliminan instancias, solo hay que modificar la lista del upstream, sin cambiar el resto de la configuración.

3. **¿Qué estrategia utiliza Nginx por defecto en este escenario?**
   Por defecto, Nginx utiliza Round Robin, que consiste en repartir las peticiones por turnos:
   - Petición 1 → Api1
   - Petición 2 → api2
   - Petición 3 → api3
   - Petición 4 → api1

   Y así sucesivamente.

En las pruebas se observó que las 12 peticiones terminaron distribuidas equitativamente entre las instancias.

4. **¿Qué ocurre al detener `api2`?**

- Aunque `api2` se detenga, el sistema sigue funcionando. Cuando Nginx intenta enviarle una petición y detecta que no responde, redirige esa solicitud a otra instancia disponible.

Desde la perspectiva del usuario, la aplicación continúa funcionando normalmente. Lo único que cambia es que ahora el trabajo se reparte entre `api1` y `api3`, por lo que hay menos capacidad disponible, pero el servicio sigue estando operativo.

5. **¿Qué componente podría convertirse en un punto único de falla?**

   - Nginx.

   Todas las peticiones pasan primero por él, por lo que si Nginx se cae, ninguna solicitud puede llegar a las APIs, aunque estas sigan funcionando perfectamente.

   Es como tener varias cajas registradoras en un supermercado, pero una sola puerta de entrada: si la puerta se bloquea, nadie puede llegar a las cajas.

6. **¿Por qué este diseño representa escalamiento horizontal?**

   - Porque para aumentar la capacidad no se hace más potente una API existente, sino que se agregan más copias de la misma aplicación. Por ejemplo, pasar de tres instancias a cinco significa que habrá más servidores disponibles para atender peticiones al mismo tiempo.

   Esto es escalamiento horizontal, pues crece agregando más nodos en lugar de aumentar los recursos de uno solo.

7. **¿Qué tendría que cambiar para tener cinco instancias?**

   - Simplemente habría que agregar los dos servicios de la API (`api4` y `api5`) en el `docker-compose.yml` y las líneas server api4:5000; y server api5:5000; en el nginx.conf

   Después de reiniciar Docker Compose, Nginx comenzaría a repartir peticiones entre las cinco instancias.

   Lo interesante es que el cliente no tendría que cambiar nada: seguiría accediendo a `localhost:8080`, mientras que Nginx se encargaría de distribuir el tráfico entre todas las APIs disponibles.

## Reto

```
casti@MLC MINGW64 ~/workshop1-docker
$ notepad api/app.py
```

```
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

@app.get("/health")
def health():
    return jsonify({
        "status": "ok",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.45kB 0.0s done
#1 DONE 0.0s

#2 [api3 internal] load build definition from Dockerfile
#2 transferring dockerfile: 204B 0.0s done
#2 DONE 0.0s

#3 [api2 internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.3s

#4 [api3 internal] load .dockerignore
```

...

```
#16 [api3] resolving provenance for metadata file
#16 DONE 0.0s
[+] up 7/7
 ✔ Image workshop1-docker-api1         Built
 ✔ Image workshop1-docker-api2         Built
 ✔ Image workshop1-docker-api3         Built
 ✔ Container workshop1-docker-nginx-1  Running
 ✔ Container workshop1-docker-api1-1   Started
 ✔ Container workshop1-docker-api2-1   Started
 ✔ Container workshop1-docker-api3-1   Started
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ docker compose ps
NAME                      IMAGE                     COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop1-docker-api1-1   workshop1-docker-api1     "python app.py"          api1      41 seconds ago   Up 29 seconds   5000/tcp
workshop1-docker-api2-1   workshop1-docker-api2     "python app.py"          api2      41 seconds ago   Up 29 seconds   5000/tcp
workshop1-docker-api3-1   workshop1-docker-api3     "python app.py"          api3      41 seconds ago   Up 29 seconds   5000/tcp
workshop1-docker-nginx-1  nginx:alpine             "/docker-entrypoint.…"   nginx     16 minutes ago   Up 16 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ cat api/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

@app.get("/health")
def health():
    return jsonify({
        "status": "ok",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

casti@MLC MINGW64 ~/workshop1-docker
$ curl http://localhost:8080/health
{"instance":"api-1","status":"ok"}
```

```
casti@MLC MINGW64 ~/workshop1-docker
$ for i in {1..12}; do curl -s http://localhost:8080/health; echo; done
{"instance":"api-3","status":"ok"}

{"instance":"api-2","status":"ok"}

{"instance":"api-1","status":"ok"}

{"instance":"api-3","status":"ok"}

{"instance":"api-2","status":"ok"}

{"instance":"api-1","status":"ok"}

{"instance":"api-3","status":"ok"}

{"instance":"api-2","status":"ok"}

{"instance":"api-1","status":"ok"}

{"instance":"api-3","status":"ok"}

{"instance":"api-2","status":"ok"}

{"instance":"api-1","status":"ok"}
```

Antes de entrar a ver cómo podría utilizarse en una estrategia de `health check`, hay que entender qué es un `health check`. Y es que este último es una consulta periódica y automática a un endpoint que el servicio expone solo para reportar su propio estado. Este

endpoint no devuelve datos del negocio: solo dice si la instancia está viva y en condiciones de atender. Por eso `/health` debe ser liviano y responder rápido.

En este taller el chequeo que hace Nginx es pasivo porque se entera de que un backend está caído solo cuando una petición real falla, y recién ahí lo saca del upstream. Eso fue lo que ocurrió al detener api2. El chequeo activo (consultar /health cada cierto tiempo aunque no haya tráfico) no existe en Nginx open source; de eso se encargaría otra herramienta, como el bloque healthcheck: de Docker Compose.

Concretamente, en este proyecto la estrategia sería algo así como configurar en el `docker-compose.yml` un bloque healthcheck: en cada servicio api, que ejecute una petición a `http://localhost:5000/health` cada 10 segundos. Si la instancia responde 200 OK, Docker la marca como healthy y sigue en rotación; si falla tres veces seguidas, la marca como unhealthy, y a partir de ahí se le puede indicar a Nginx que no reciba tráfico o al orquestador que la reinicie. Así el endpoint deja de ser solo informativo y pasa a ser el criterio que decide si una instancia entra o sale del balanceo.